

ADA-05

Spec-Driven Feature

Ingeniería de Software asistida por IA · UADY

Spec-Driven Feature

1. Objetivo

Construir una feature pequeña siguiendo requirements → specification → architecture → tasks → implementation → tests. El foco es demostrar que una especificación estructurada puede convertirse en trabajo ejecutable por un coding agent.

2. Restricciones

- No se proporciona repositorio base: todo se construye desde cero.
- Usa herramientas gratuitas.
- Usa Antigravity CLI (agy).
- No se requiere API de pago.
- El desarrollador debe revisar y validar el código generado.
- Mantén un AI Usage Log durante todo ADA-05. Registra las interacciones de IA que influyan en requisitos, arquitectura, tareas, implementación, pruebas o revisión.

3. Feature

Implementar una función de Customer Search que permita buscar clientes por nombre o correo electrónico. Debe soportar búsqueda parcial, validación y pruebas automatizadas. Puede ser una CLI local o una API HTTP local.

4. Stack recomendado

Componente	Opción
Lenguaje	Python 3.11+
Tests	pytest
Versionado	Git
Agent	Antigravity CLI (agy)
Persistencia	Memoria o JSON local
HTTP	FastAPI opcional

La solución mínima puede ser Python + CLI + pytest, sin base de datos.

5. Fase 1 — REQUIREMENTS.md

Antes de pedir código, define qué necesita el usuario. REQUIREMENTS.md es la fuente de verdad de los requisitos del producto: cada requisito se define una sola vez y recibe un identificador estable.

Regla clave: los requisitos FR/NFR se definen aquí. SPEC.md los referencia por ID; no los vuelve a copiar.

```
# Requirements – Customer Search

## User Story
As a user,
I want to search customers by name or email,
so that I can quickly find the customer record I need.

## Functional Requirements
FR-01:
FR-02:
FR-03:
FR-04:
FR-05:
FR-06:

## Non-Functional Requirements
NFR-01:
NFR-02:
NFR-03:

## Open Questions
Q-01:
Q-02:

## Constraints / Assumptions
C-01:
A-01:
```

6. Fase 2 — SPEC.md

SPEC.md es la fuente de verdad del comportamiento verificable de la feature. Parte de REQUIREMENTS.md y transforma los requisitos en alcance, reglas, validaciones, criterios de aceptación y escenarios de prueba. No redefine FR/NFR.

```
# Customer Search Feature

## Goal

## Requirements Covered
- FR-01
- FR-02
- FR-03
- FR-04
- FR-05
- FR-06
- NFR-01
- NFR-02
- NFR-03

## Scope

## Out of Scope

## Domain Model

## Search Rules

## Validation Rules

## Error Handling

## Acceptance Criteria
```

```
## Test Scenarios

## Constraints

## Open Questions
```

No dejes secciones vacías. Si no aplica, escribe N/A y justifica. Los IDs incluidos en Requirements Covered deben existir en REQUIREMENTS.md.

7. Fase 3 — ARCHITECTURE.md

```
# Architecture

## Overview

## Components

## Responsibilities

## Data Flow

## Interfaces

## Error Handling

## Testing Strategy

## Dependencies

## Design Decisions

## Trade-offs
```

Incluye un diagrama Mermaid o ASCII si mejora la comunicación.

8. Fase 4 — TASKS.md

```
# Tasks

## T-01 Project setup
- Goal:
- Files:
- Acceptance:
- Verification:

## T-02 Domain model
- Goal:
- Files:
- Acceptance:
- Verification:

## T-03 Search logic
- Goal:
- Files:
- Acceptance:
- Verification:

## T-04 Validation and errors
- Goal:
- Files:
- Acceptance:
- Verification:

## T-05 Tests
- Goal:
- Files:
- Acceptance:
```

```
- Verification:

## T-06 Documentation
- Goal:
- Files:
- Acceptance:
- Verification:
```

Cada tarea debe ser pequeña, observable y verificable. TASKS.md no puede introducir requisitos nuevos: ejecuta lo definido en REQUIREMENTS.md y especificado en SPEC.md.

9. Fase 5 — AGENTS.md

```
# Agent Instructions

## Project Rules
- Read REQUIREMENTS.md before implementing.
- Read SPEC.md before implementing.
- Read ARCHITECTURE.md before architectural changes.
- Follow TASKS.md.
- Prefer small, focused changes.
- Do not invent business requirements.
- Do not modify REQUIREMENTS.md or SPEC.md to make tests pass.
- Do not delete or weaken tests.
- Do not add dependencies without justification.

## Validation
- Run pytest before and after changes.
- Add tests for new behavior.
- Report changed files and test results.
- Stop and ask for clarification if requirements conflict.

## Definition of Done
- Relevant tests pass.
- Acceptance criteria are covered.
- No unrelated files are changed.
- Documentation reflects final behavior.
```

10. Fase 6 — Git baseline

```
git init
git add .
git commit -m "spec: define customer search feature"
```

El commit debe contener REQUIREMENTS.md, SPEC.md, ARCHITECTURE.md, TASKS.md, AGENTS.md y la estructura inicial. Todavía no implementes.

11. Fase 7 — Antigravity

```
agy

We are implementing the Customer Search feature.

Before changing code:
1. Read REQUIREMENTS.md.
2. Read SPEC.md.
3. Read ARCHITECTURE.md.
4. Read TASKS.md.
5. Read AGENTS.md.
6. Inspect the current repository.
7. Identify the next incomplete task.
8. Propose the implementation approach.
9. Implement only that task.
10. Run relevant tests.
11. Report changes, verification, and unresolved issues.

Do not invent business requirements.
```

Do not modify REQUIREMENTS.md or SPEC.md to accommodate an implementation.

Trabaja tarea por tarea. Después de cada una, revisa cambios y pruebas. Si el agente detecta una contradicción entre REQUIREMENTS.md y SPEC.md, debe detenerse y pedir aclaración; no debe decidir cuál ignorar.

12. Fase 8 – Trazabilidad

La trazabilidad conecta el requisito definido una sola vez en REQUIREMENTS.md con su comportamiento verificable en SPEC.md y con la implementación.

Requirement	SPEC / AC	Task	Files	Test	Status	Notes
FR-01						
FR-02						
FR-03						
FR-04						
FR-05						
FR-06						
NFR-01						
NFR-02						
NFR-03						

13. Fase 9 – Pruebas

```
pytest -v
```

Cada criterio de aceptación importante definido en SPEC.md debe tener una prueba automatizada o una explicación de cómo se verifica. La prueba debe poder rastrearse hasta el Requirement correspondiente.

14. Fase 10 – Revisión humana

- ¿La implementación cumple REQUIREMENTS.md y SPEC.md?
- ¿SPEC.md referencia requisitos existentes sin redefinirlos?
- ¿Se inventaron reglas?
- ¿Hay código innecesario?
- ¿Se modificaron archivos no relacionados?
- ¿Los tests prueban comportamiento?
- ¿ARCHITECTURE.md coincide con el código?
- ¿La documentación está actualizada?

14.1. AI Usage Log durante ADA-05

Continúa el registro iniciado o crea un archivo AI_USAGE_LOG.md en el repositorio. Debe cubrir, como mínimo, interacciones relevantes de requisitos/especificación y de implementación/pruebas.

El log debe permitir reconstruir qué propuso la IA, qué decisión tomó el estudiante y qué cambió en el producto. No copies conversaciones completas: selecciona evidencia significativa y resume la contribución cuando sea suficiente.

Como mínimo, incluye 4 entradas significativas en ADA-05, distribuidas entre diferentes etapas del flujo spec-driven.

15. Definition of Done

Condición	¿Cuándo puede marcarse como OK?
Requirements definidos	REQUIREMENTS.md contiene requisitos funcionales, no funcionales, preguntas abiertas y, cuando apliquen, restricciones/supuestos. Cada requisito tiene un ID estable; los usados para implementar son claros y verificables.
SPEC.md completo	SPEC.md referencia los requisitos mediante Requirements Covered y define alcance, reglas, validaciones, errores, criterios de aceptación, escenarios de prueba y restricciones. No duplica ni redefine FR/NFR.

ARCHITECTURE.md completo	Describe componentes, responsabilidades, flujo, interfaces, manejo de errores, estrategia de pruebas y decisiones de diseño de la solución implementada.
TASKS.md completo	El trabajo fue dividido en tareas pequeñas y verificables. Cada tarea indica al menos objetivo, archivos involucrados, aceptación y forma de verificación. No introduce requisitos nuevos.
AGENTS.md creado	Contiene las instrucciones del coding agent: qué documentos leer, reglas del proyecto, restricciones, validación y cuándo detenerse/pedir aclaración.
Feature implementada	Customer Search funciona de acuerdo con REQUIREMENTS.md y SPEC.md; no basta con que exista código.
Tests automatizados	Existen pruebas automatizadas relevantes y pueden ejecutarse con pytest.
Acceptance criteria verificados	Cada criterio de aceptación definido en SPEC.md fue comprobado mediante una prueba automatizada o evidencia verificable.
Trazabilidad completa	Puede seguirse Requirement → Acceptance Criterion → Task → Implementation → Test en docs/traceability.md.
Revisión humana documentada	El estudiante revisó el trabajo generado con IA y documentó qué encontró, qué modificó y/o por qué aceptó la solución.
AI Usage Log completo y trazable	AI_USAGE_LOG.md registra interacciones significativas con IA y permite relacionarlas con decisiones y cambios reales del proyecto.

16. Estructura de entrega

```

ada-05-spec-driven-feature/
├── README.md
├── REQUIREMENTS.md
├── SPEC.md
├── ARCHITECTURE.md
├── TASKS.md
├── AGENTS.md
├── AI_USAGE_LOG.md
├── src/
├── tests/
├── docs/
│   └── traceability.md
├── results/
│   └── agent-report.md

```

17. agent-report.md

```

# Agent Report

## Agent / Version

## Initial Context

## Task Sequence

### T-01
What the agent did:
Human review:
Tests:

### T-02
What the agent did:
Human review:
Tests:

## Problems Encountered

## Human Interventions

## Requirement / Specification Changes
If any, explain why, who approved them, and which artifacts were updated.

## Final Verification

```

El agent-report.md resume la secuencia de trabajo del agente; AI_USAGE_LOG.md documenta las interacciones de IA que influyeron en decisiones. Ambos deben ser consistentes entre sí. Si cambia un requisito, primero se actualiza REQUIREMENTS.md y después se revisan SPEC.md, TASKS.md, trazabilidad y pruebas afectadas.

18. Reflexión

1. ¿Qué diferencia hay entre requirement, specification y prompt?
2. ¿Qué información fue indispensable antes de programar?
3. ¿Qué decisiones debieron resolverse antes de programar?
4. ¿Qué errores evitó SPEC.md?
5. ¿Qué problema evita separar REQUIREMENTS.md de SPEC.md?
6. ¿Qué papel tuvo AGENTS.md?
7. ¿Qué cambió durante la revisión humana?
8. ¿La arquitectura coincidió con el código final?
9. ¿Qué requisito fue más difícil de probar?
10. ¿Qué mejorarías en tu proceso spec-driven?
11. ¿Por qué SPEC.md puede funcionar como contrato verificable sin duplicar los requisitos?
12. ¿Qué decisiones importantes aparecen en tu AI Usage Log y cómo cambió tu criterio después de revisar las sugerencias de la IA?

19. Entregables finales

Entrega	Evidencia
ADA-05	Repositorio Git
Requirements	REQUIREMENTS.md
Specification	SPEC.md
Architecture	ARCHITECTURE.md
Tasks	TASKS.md
Agent instructions	AGENTS.md
Tests	tests/
Traceability	docs/traceability.md
Agent evidence	results/agent-report.md
README	Instalación, ejecución y pruebas
AI Usage Log	AI_USAGE_LOG.md o sección equivalente claramente identificada

20. Regla académica de IA

La IA es parte central del ADA. Sin embargo, entregar código generado por un agente sin poder explicar requisitos, especificación, arquitectura, decisiones y pruebas no demuestra la competencia. Debes poder explicar las partes relevantes y distinguir decisiones humanas de sugerencias o implementaciones del agente. El AI Usage Log forma parte de la evidencia académica y deberá ser coherente con los artefactos, commits, pruebas y decisiones reportadas.